

GRS Breast Cancer Awareness Campaign **Platform**

Phase 1 & Phase 2 Architectural & Technical Briefing

Senior Software Engineer Mentor: Antigravity AI

Date: July 11, 2026

Prepared for: Next.js Junior Engineer (Stakeholder)

A complete conceptual guide explaining the full Next.js 15, PostgreSQL, Prisma ORM, and Auth.js stack, request lifecycles, authentication flows, and project schemas designed in Phase 1 & 2.

1. Why was Phase 1 and 2 required? (Yeh Phase Kyun Chahiye Tha?)

Kisi bhi real-world enterprise website ko banane se pehle hume blueprint aur base setup ki zaroorat hoti hai.

Real-life example: Agar hum bina blueprint ke building banana shuru kar denge, toh deewar khadi karne ke baad pata chalega ki bathroom ka pipeline lagana toh bhool hi gaye!

Phase 1 (PRD) aur Phase 2 (Base Setup & Auth) isiliye required the taki hum project ke goals aur technology stack ko clear kar sakein aur ek secure, scalable setup ready kar sakein jisme roles-based access control ho (Doctor, Patient, NGO, Volunteer, etc. ke beech safe boundary).

2. What problem does it solve? (Yeh Kya Problem Solve Karta Hai?)

- * Scope Creep Se Bachata Hai: Client aur developers ko pehle se pata hota hai ki project me kya-kya banega.

- * Fraudulent Crowdfunding Se Bachata Hai: Base DB schema me direct hospital account integration aur NGO endorsements validation flows include kiye hain, taki patient ke naam par fake donations na uthaye ja sakein.

- * Dynamic Profiles: Har role (Patient, Doctor, NGO) ke paas wahi options honge jo unke liye designed hain. Ek guest user sensitive administrative routes ya clinical panels access nahi kar payega.

3. What exactly did we implement? (Humne Kya-Kya Implement Kiya?)

- * PRD Document: Pooray business model aur target audience ka blueprint.

- * Next.js 15 setup using TypeScript & Tailwind CSS v4.

- * shadcn/ui custom setup jisme rose-pink thematic color design tokens configuration feed ki taaki breast cancer awareness theme (pink ribbon concept) reflect ho.

- * Prisma v7 PostgreSQL modeling jiske models user sessions, campaigns aur certificates ko connect karte hain.

- * Auth.js v5 credentials adapter flow: Passwords hash ho kar securely DB me check hote hain.

- * Layout components: Responsive Navbar, Footer, Page view controllers, login/register visual components.

4. Which files did we create? (Humne Kaunsi Files Banayi?)

- * prisma/schema.prisma (Database tables structure design)

- * src/lib/db.ts (Prisma PostgreSQL singleton client connection)

- * src/auth.ts, src/auth.config.ts, src/types/next-auth.d.ts (Auth.js core files)

- * src/middleware.ts (Global secure routes guard redirector)

- * src/app/layout.tsx, src/app/page.tsx, src/components/layout/Navbar.tsx (Layout templates and landing view)

- * src/app/actions/auth.ts (Server action registering users in database)

- * src/app/login/page.tsx, src/app/register/page.tsx, src/app/dashboard/page.tsx (Interactive views)

5. Why were those files needed? (Yeh Files Kyun Zaroori Thi?)

- * schema.prisma is required so PostgreSQL knows the structure of User and Campaign tables.

- * db.ts prevents connection pool crash by caching the PrismaClient instance.

- * auth.ts & auth.config.ts handle credential validations and token encryptions.

- * middleware.ts guards folder paths (/admin, /dashboard) at edge nodes before requests hit server logic.

- * Navbar.tsx is dynamic; it shows login links if logged out, or avatar dropdowns + role badges if logged in.

- * Server Actions (actions/auth.ts) bypass write APIs by executing secure node queries directly on forms

submit.

6 & 7. Project Folder Structure Explained (Folder Structure Ki Kahani)

Chalo, folder structure ko ek simple kitchen ki tarah samajhte hain:

- Kitchen (Workspace Root): Isme hamara main setup hai.
- package.json: Kitchen ki inventory list aur recipes (dependencies and build scripts).
- .env: Masala box (environment variables like secret keys and DB urls).
- prisma/ folder: Fridge jisme database ki guidelines (schema) rakhi hain.
- src/ folder: Cooking Area jahan saara real code cooking hota hai:
 - * app/: Alag-alag dishes (Next.js route pages like /login, /register, /dashboard).
 - * components/: Raw ingredients jise bar-bar reuse karte hain (buttons, input fields, Navbar).
 - * lib/: Hand-blenders aur tools (db.ts connector).
 - * types/: Items specifications tags (custom typescript NextAuth roles declarations).

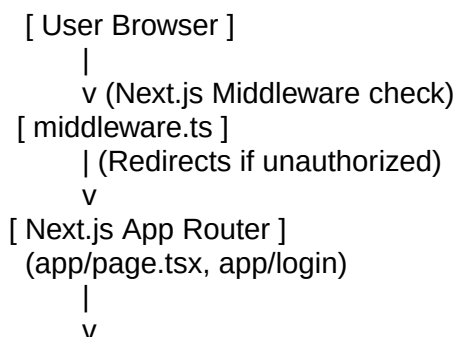
8. Technologies Used & Simplified Explanation

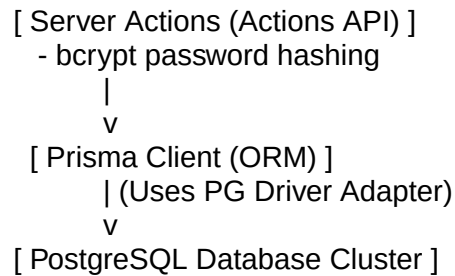
- * Next.js 15: Ek Framework (Car Chassis). Yeh server-side rendering (SSR), client-side transitions, routes, aur API functions ko handle karta hai.
- * PostgreSQL: Ek Relational Database (Log Book/Warehouse). Hamare data (Users, Donations, Campaigns) ko tables me securely store karta hai.
- * Prisma v7: Ek ORM (Object-Relational Mapping) - Ek Translator. Hum JavaScript likhte hain, aur Prisma use SQL query me convert karke Postgres ko samjhata hai.
- * Auth.js v5: Security Guard. Cookies verify karke check karta hai ki user logged in hai ya nahi, aur check karke role allow karta hai.
- * Tailwind CSS v4: Paint and Style. Pre-coded utility class name attributes (flex, text-primary, p-8) provide karta hai.
- * shadcn/ui: Ready-made Furniture. Pehle se styled built-in components (cards, dialog boxes) jise hum as-is install karke use karte hain.

9. How all these technologies work together (Ek Saath Kaise Kaam Karta Hai?)

Browser (User) -> Clicks 'Sign Up' -> Next.js serves src/app/register/page.tsx.
User enters fields -> Clicks Button -> Next.js Form triggers registerUserAction (Server Action).
Server Action calls bcryptjs to hash password -> Sends command to Prisma Client (src/lib/db.ts).
Prisma translates JSON to SQL Command (e.g. INSERT INTO User...) -> Sends to PostgreSQL database.
PostgreSQL saves the record -> Sends 'OK' back to Prisma -> Server Action redirects user to '/login'.

10. Project Architecture Diagram





11. Request Flow (Browser to Database)

Analogy: Ek restaurant me khana order karne ka process:

1. Customer (Browser) waiter (Next.js App Router) ko order deta hai: "Please fetch my active campaigns."
2. Waiter guard (Middleware) ke paas jata hai: "Yeh user authenticated hai kya?" Guard cookie status check karke pass deta hai.
3. Waiter chef (Server Action / Next.js backend handler) ke paas order deta hai.
4. Chef raw ingredients supplier (Prisma ORM) se bolta hai: "DB warehouse se campaigns details nikal kar do."
5. Prisma warehouse clerk (pg driver adapter) ko bolta hai aur SQL query execute karwata hai.
6. Warehouse (PostgreSQL Database) data deta hai -> Prisma use JavaScript objects me maps karta hai -> Chef cooks layout view -> Waiter serves delicious UI back to Browser!

12. Authentication Flow (Securing Session)

1. User inputs email and password on /login screen.
2. NextAuth signIn("credentials") is triggered.
3. Server executes authorize() handler in src/auth.ts.
4. Database is queried. If user is found, bcrypt.compare compares incoming text with stored hash.
5. If valid, NextAuth encrypts user details (id, role) into a secure JSON Web Token (JWT).
6. JWT is stored inside a secure HTTP-Only cookie in the browser.
7. For future actions, middleware.ts decrypts the cookie to verify role levels (e.g., patient cannot load NGO pages).

13. Database Schema and Relationships (Rishte in Database)

- * User (id, email, passwordHash, role) represents accounts.
- * Profile (userId, phone, docLicense, ngoRegNum) holds details. Every User has exactly ONE Profile (1-to-1 relationship).
- * Campaign (id, patientId, title, fundingGoal) has details. One User (Patient) can have multiple Campaigns (1-to-many relationship).
- * Donation (id, campaignId, donorId, amount) records money flow. Links many donations to one campaign (1-to-many) and many donations to one User (donor) (1-to-many).
- * Certificate (id, recipientId, verificationHash) records credentials earned by Volunteers (1-to-many relationship).

14. What happens when a user registers? (Sign Up Ke Piche Ki Kahani)

1. User selects role (e.g., Doctor) -> Fills inputs (name, license number).
2. UI calls registerUserAction(formData).
3. db.user.findUnique checks if email exists. If yes, returns error.
4. bcrypt.hash encrypts the password.
5. db.user.create adds record to User table.

6. db.profile.create links a Profile entry containing 'VerificationStatus: PENDING' and docLicense data.
7. Redirects client to /login page with a success banner.

15. What happens when a user logs in? (Log In Ke Piche Ki Kahani)

1. User types credentials. Auth.js calls authorize().
2. Query checks User by email. Compares password hashes.
3. If matches, Auth.js jwt() callback serializes user.id and user.role.
4. NextAuth session() callback maps these variables into dynamic request session contexts.
5. Next.js router routes browser to '/dashboard'.
6. /dashboard/page.tsx reads user.role and renders customized dashboards.

16. What happens internally when the application starts? (Boot Process)

1. 'npm run dev' or 'next start' boots the Node.js server.
2. Next.js compiles Tailwind CSS v4 directives from globals.css.
3. Loads configuration maps from next.config.ts and prisma.config.ts.
4. Reads .env configuration parameters.
5. Prisma loads connection pool with pg driver adapter and connects to PostgreSQL.
6. Server-side App Router routes register pages. Edge node runtime registers middleware interceptors.

17. Why PostgreSQL & Prisma? (Database Aur Translator Ka Choice)

- * PostgreSQL: A highly robust, open-source relational database. For monetary platforms, we need transactions safety (ACID compliance) so donations are never lost or double-debited. Postgres handles complex relational grids (Users -> Campaigns -> Donations) smoothly.
- * Prisma: It provides complete TypeScript auto-completion. If we write a typo like 'user.eamil', Prisma compiler immediately errors before deployment. Migrations are managed cleanly.

18. Why Next.js 15? (Framework Ka Choice)

Next.js 15 is the latest industry standard. It merges frontend and backend API layers into one single workspace.

- * Server Components: Loads campaign lists on server, rendering super fast SEO-friendly pages.
- * Server Actions: Safe backend forms handlers without writing separate REST controller routing APIs.
- * Edge Middleware: Perform session checking at router gateway nodes directly, making redirection immediate.

19. What has NOT been built yet? (Kya Baki Hai?)

- * Razorpay donation payment handlers & checkout APIs.
- * Cloudinary file upload logic (PDF bills, doctor verification sheets).
- * Asynchronous PDF certificate builder script containing QR verification graphics.
- * Campaign moderation pipeline (Super-admin verification consoles).

20 & 21. What Phase 3 contains & Overall Roadmap

Phase 3 covers 'Core Awareness CMS & Event Scheduling':

- Blog management interface (Doctors write educational blogs).
- Interactive physical breast check layouts with countdown guidelines.
- RSVP panels for scheduled webinars.

Overall Roadmap:

- Phase 2: Setup & Authentication (Done).
- Phase 3: CMS Awareness & BSE Guides.
- Phase 4: Fundraising & Razorpay Integration.
- Phase 5: QR PDF Certificate Engine.
- Phase 6: Admin Dashboard Command Panels.
- Phase 7: Polish, Audits, and Production Release.

22. How the platform will work for everyone (Kaun Kaise Use Karega?)

- * Patients: Sign up, input details. Launch fundraiser campaign, request NGO endorsement, monitor donation counters. Request direct hospital disbursements.
- * Doctors: Register, wait for Admin approval. Publish cancer check guidelines, schedule webinars, review medical sheets for patients.
- * Researchers: View statistics panels, read updates, check regional trends.
- * Donors: Search active endorsed campaigns, pay securely via Razorpay, obtain 80G tax benefits.
- * Volunteers: Complete awareness tests, attend webinars, download verified QR certificates.
- * Admins: command center controls: approve/reject Doctors, NGOs, Campaigns, process hospital payout clearances.